



# 10장 1강 : Timeout/Rate Limit/Retry/Fallback

## 학습 목표

- API 오류를 재시도 가능 여부에 따라 분류할 수 있습니다.
- OpenAI Python SDK의 기본 retry와 timeout 동작을 설명할 수 있습니다.
- `APIConnectionError`, `APITimeoutError`, `RateLimitError`, `APIStatusError`를 구분할 수 있습니다.
- exponential backoff와 jitter를 구현할 수 있습니다.
- 인증/잘못된 요청처럼 재시도하면 안 되는 오류를 구분할 수 있습니다.
- 실패 시 모델/기능/사람 처리로 이어지는 fallback ladder를 설계할 수 있습니다.

## 선수지식 확인

- Responses API 기본 호출
- Python `try / except`, 함수, `time.sleep()`
- 7-3강의 오류 분류와 제한된 재시도
- 8장의 Tool 실패 결과와 `retryable` 개념

## 튜터 한마디

안정적인 API 호출은 오류를 숨기는 코드가 아닙니다. 잠깐 기다리면 해결될 오류와 설정을 고쳐야 하는 오류를 구분하고, 같은 요청을 무한 반복하지 않는 코드입니다. SDK가 이미 재시도하는 범위도 있으므로 수동 retry를 추가할 때 중복 정책을 먼저 확인하세요.

## 목차

1. 안정적인 API 호출이란 무엇인가요?
2. 오류 처리의 큰 그림
3. 재시도 가능한 오류와 불가능한 오류
4. SDK 기본 retry와 timeout
5. Exponential Backoff와 Jitter
6. Fallback Ladder 설계
7. 기본 실습: 로컬 Retry 함수 만들기
8. 연습 문제: 재시도 정책 판단
9. 추가 실습: Responses API Wrapper 구현
10. 실제 활용과 다음 강의 연결
11. 자주 생기는 오해
12. 이해도 점검
13. 이번 강의 요약

## 1. 안정적인 API 호출이란 무엇인가요?

네트워크 API는 정상적인 코드에서도 일시적으로 실패할 수 있습니다.

네트워크 끊김 서버 응답 지연 Rate Limit 일시적인 5xx 오류 잘못된 API key 지원하지 않는 요청 파라미터

안정적인 호출 코드는 다음 질문에 답합니다.

- 이 오류는 다시 시도하면 해결될 가능성이 있나요?
- 얼마나 기다렸다가 다시 시도할까요?
- 최대 몇 번 시도할까요?
- 재시도하는 동안 같은 부작용이 중복되지 않나요?
- 모두 실패하면 사용자에게 무엇을 안내할까요?
- 운영자가 원인을 찾을 수 있도록 무엇을 기록할까요?

API 오류 분류

### API 오류를 먼저 분류합니다

같은 retry 정책을 모든 오류에 적용하지 않습니다.

| 종류           | 예시            | 기본 대응              |
|--------------|---------------|--------------------|
| 입력 인증        | 400, 401, 403 | 요청 수정 또는 즉시 중단     |
| Rate limit   | 429           | jitter가 있는 backoff |
| 일시 서버 오류     | 5xx           | 제한적 retry          |
| 네트워크 Timeout | 연결 실패         | retry 또는 fallback  |
| 업무 실패        | Schema 도구 오류  | 복구 정책 또는 사용자 안내    |

Retry는 오류를 없애는 방법이 아닙니다. 일시적인 실패에서 회복할 기회를 주는 제한된 정책입니다.

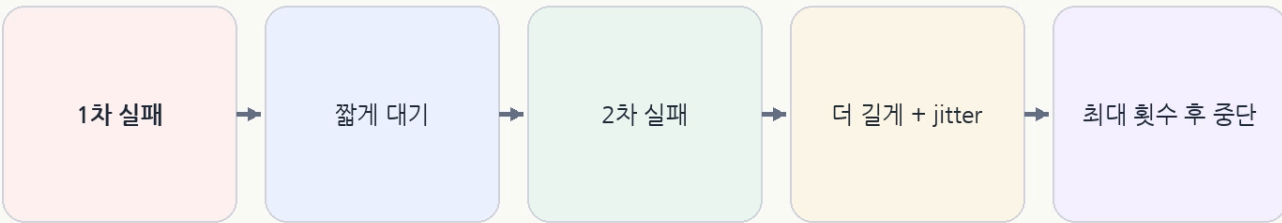
## 2. 오류 처리의 큰 그림

API 요청 -> 성공: 응답 검증 후 반환 -> 실패 |─ retryable: backoff 후 제한 횟수 재시도 |─ non-retryable: 즉시 중단하고 설정/입력 오류 반환 └─ 횟수 소진: fallback 또는 사용자 안내

Retry와 Backoff 흐름

### Exponential Backoff + Jitter

실패 직후 같은 속도로 몰아치지 않습니다



실패 요청도 rate limit에 포함될 수 있습니다. 재시도 횟수와 총 대기 시간을 제한합니다.

오류 처리에는 세 가지 층이 있습니다.

SDK 층

OpenAI Python SDK가 일부 네트워크/HTTP 오류를 기본 재시도합니다.

애플리케이션 층

업무 단위 retry, fallback, circuit breaker, 사용자 메시지를 관리합니다.

Tool/외부 시스템 층

DB, 사내 API, 결제 시스템 등 각 외부 서비스의 timeout과 retry 정책을 관리합니다.

여러 층에서 모두 긴 retry를 사용하면 실제 호출 횟수와 지연시간이 예상보다 크게 늘 수 있습니다.

SDK 3회 × 애플리케이션 3회 × Tool API 3회 -> 최악의 경우 최대 27회 시도 가능

따라서 각 층의 책임을 문서화해야 합니다.

3. 재시도 가능한 오류와 불가능한 오류

대표적으로 재시도를 고려하는 오류

| 오류                  | 이유                       |
|---------------------|--------------------------|
| 네트워크 연결 실패          | 일시적 연결 문제일 수 있습니다.       |
| timeout             | 서버나 네트워크가 잠시 느릴 수 있습니다.  |
| 408 Request Timeout | 요청 처리 시간이 초과되었습니다.       |
| 409 Conflict        | 일시적 충돌일 수 있습니다.          |
| 429 Rate Limit      | 잠시 기다리면 할당량이 회복될 수 있습니다. |
| 5xx                 | 서버의 일시적 장애일 수 있습니다.      |

재시도 전에 설정을 고쳐야 하는 오류

| 오류                    | 처리                  |
|-----------------------|---------------------|
| 잘못된 API key           | secret과 권한 확인       |
| 400 Bad Request       | 요청 payload와 파라미터 수정 |
| 지원하지 않는 model/feature | 모델 문서 확인 후 설정 변경    |
| Pydantic 업무 규칙 실패     | 입력 또는 Prompt 정책 수정  |
| permission denied     | 사용자 권한 확인           |

⚠ Rate Limit에 걸린 실패 요청도 사용량 제한 계산에 영향을 줄 수 있습니다. 기다림 없이 빠르게 반복하면 상황이 악화될 수 있습니다.

HTTP status만으로 충분하지 않은 경우

응답 body의 오류 코드, **Retry-After** header, 외부 Tool의 **retryable** 필드도 확인합니다. SDK가 제공하는 예외 객체의 status code와 request ID를 로그에 남기면 운영자가 원인을 찾기 쉽습니다.

4. SDK 기본 retry와 timeout

Retry 책임 분리

### Retry의 책임은 한 계층에 모읍니다

SDK 기본 재시도와 애플리케이션 재시도를 겹치면 실제 호출 횟수가 예상보다 초과될 수 있습니다.

**선택 A: SDK가 담당**  
OpenAI SDK  
  
기본 max\_retries=2  
연결 오류·408·409·429·5xx  
짧은 exponential backoff

**선택 B: 앱 wrapper가 담당**  
OpenAI(max\_retries=0)  
  
업무별 횟수·deadline  
full jitter-fallback  
관찰 가능한 로그

둘을 무심코 중첩하지 않습니다

OpenAI Python SDK는 연결 오류, 408, 409, 429, 5xx를 기본적으로 두 번 재시도합니다. 애플리케이션에서 별도의 retry wrapper를 만들 때는 다음 중 한 방식을 명시적으로 선택하세요.

A. SDK retry를 사용하고 애플리케이션은 fallback만 담당합니다. B. OpenAI(max\_retries=0)으로 SDK retry를 끄고 애플리케이션이 횟수/deadline/로그를 담당합니다.

두 retry 계층을 무심코 중첩하면 실제 요청 수와 최악 지연시간이 급셈처럼 늘어날 수 있습니다.

OpenAI Python SDK는 일부 오류를 기본적으로 두 번 재시도합니다. 대표 대상은 연결 오류, 408, 409, 429, 5xx입니다.

기본 재시도 끄기

수업에서 수동 retry 동작을 명확히 보기 위해 SDK retry를 끌 수 있습니다.

```
from openai import OpenAI # API key는 코드에 쓰지 않고 OPENAI_API_KEY 환경 변수 또는 Colab Secrets에서 읽습니다. manual_retry_client = OpenAI( # SDK 자체 재시도를 끕니다. # 이후 애플리케이션 wrapper가 모든 retry 횟수를 관리합니다. max_retries=0, # 초 단위 timeout입니다. 실제 값은 작업과 모델 latency에 맞게 조정합니다. timeout=20.0, )
```

실제 서비스에서는 다음 두 방식 중 하나를 선택할 수 있습니다.

방식 A: SDK 기본 retry 사용 + 애플리케이션은 업무 fallback만 담당  
방식 B: SDK retry 최소화 + 애플리케이션 wrapper에서 retry를 통합 관리

Timeout

SDK의 기본 timeout은 매우 길 수 있으므로 사용자-facing 서비스에서는 명시적으로 설정하는 편이 좋습니다.

```
import httpx from openai import OpenAI # API key는 코드에 쓰지 않고 OPENAI_API_KEY 환경 변수 또는 Colab Secrets에서 읽습니다. client = OpenAI( timeout=httpx.Timeout( 60.0, # 전체 기본 timeout connect=5.0, # 서버 연결을 기다릴 최대 시간을 지정합니다. read=45.0, # 응답 데이터를 읽을 최대 시간을 지정합니다. write=10.0, # 요청 데이터를 전송할 최대 시간을 지정합니다. ), )
```

Reasoning이나 큰 멀티모달 요청은 단순 분류보다 오래 걸릴 수 있습니다. 모든 태스크에 같은 timeout을 적용하기보다 작업 유형별 기준을 정하세요.

### 5. Exponential Backoff와 Jitter

Exponential backoff는 재시도 간격을 점점 늘립니다.

기본 대기 1초 1차 실패 -> 약 1초 2차 실패 -> 약 2초 3차 실패 -> 약 4초

Jitter는 여러 client가 정확히 같은 시간에 다시 요청하지 않도록 대기 시간을 무작위화합니다.

상한 = min(max\_delay, base\_delay × 2^attempt) 대기 시간 = Uniform(0, 상한)

Exponential Backoff

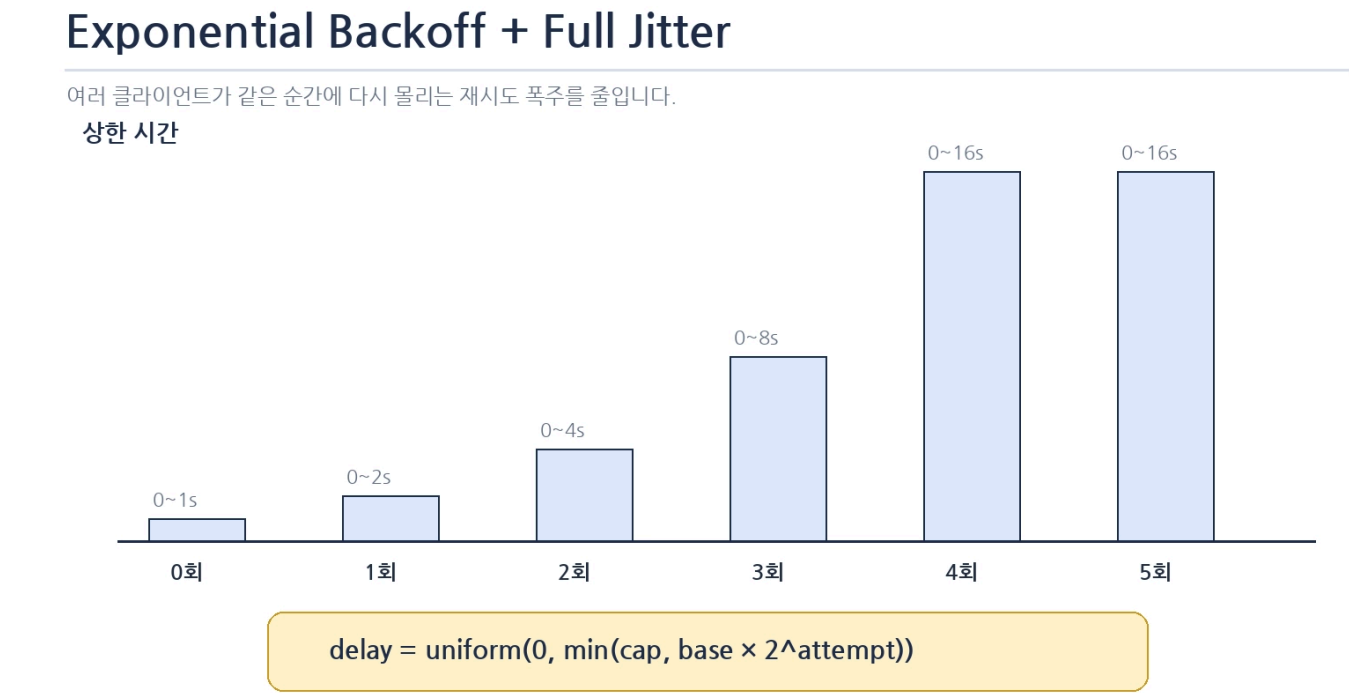


이번 교안에서는 Full Jitter를 사용합니다.

$$D_a \sim \text{Uniform}(0, \min(D_{max}, D_0 \times 2^a))$$

- $a$ : 0부터 시작하는 재시도 인덱스
- $D_0$ : 기본 대기 시간
- $D_{max}$ : 최대 대기 시간
- $D_a$ : 실제로 기다릴 무작위 시간

Full Jitter Backoff



상한은 지수적으로 커지지만, 실제 대기 시간은 0과 상한 사이에서 무작위로 선택합니다. 여러 client가 같은 시점에 실패해도 다시 같은 순간에 물릴 가능성을 줄일 수 있습니다.

#### 대기 시간 계산 함수

```
# 재시도 횟수에 따라 증가하는 상한을 계산하고 Full Jitter 대기 시간을 뽑습니다. # [비율] 로컬 계산만 수행하며 API 호출은 없습니다. # [정확성] attempt_index는 0부터 시작하며 음수는 즉시 거부합니다. # [테스트] 반환값이 0 <= delay <= upper_bound인지 여러 번 확인할 수 있습니다. import random # calculate_backoff_seconds 처리 단계를 함수로 묶어 재사용합니다. def calculate_backoff_seconds( attempt_index: int, base_delay: float = 1.0, max_delay: float = 20.0, ) -> float: """Exponential backoff 상한 안에서 Full Jitter 대기 시간을 반환합니다.""" # 남은 재시도 횟수와 오류 종류를 기준으로 계속할지 판단합니다. if attempt_index < 0: # 잘못된 입력을 조기에 중단해 이후 단계로 전파되지 않게 합니다. raise ValueError("attempt_index는 0 이상이어야 합니다.") # 조건을 검사해 정상 경로와 실패 경로를 분리합니다. if base_delay <= 0 or max_delay <= 0: # 잘못된 입력을 조기에 중단해 이후 단계로 전파되지 않게 합니다. raise ValueError("base_delay와 max_delay는 0보다 커야 합니다.") # 재시도할수록 상한은 1, 2, 4, 8초처럼 증가합니다. exponential_limit = base_delay * (2 ** attempt_index) # 정애가 길어져도 한 번에 기다리는 시간이 무한히 커지지 않도록 상한을 제한합니다. upper_bound = min(exponential_limit, max_delay) # Full Jitter: 0과 upper_bound 사이에서 실제 대기 시간을 무작위로 선택합니다. return random.uniform(0.0, upper_bound) # 각 항목을 순회하며 현재 조건에 해당하는지 확인합니다. for attempt_index in range(5): # 현재 재시도 전에 기다릴 시간을 계산합니다. delay = calculate_backoff_seconds(attempt_index) # 지수 증가와 최대 대기 시간을 함께 반영한 상한을 계산합니다. upper_bound = min(20.0, 1.0 * (2 ** attempt_index)) print(attempt_index, round(delay, 3), "upper_bound=", upper_bound) assert 0.0 <= delay <= upper_bound
```

#### 서버가 Retry-After를 제공하는 경우

가능하면 서버가 제안한 대기 시간을 우선 고려합니다. 단, 무제한으로 긴 값을 그대로 기다리지 않도록 애플리케이션 최대 대기 정책도 둡니다.

### 6. Fallback Ladder 설계

모든 재시도도 실패했을 때의 다음 경로를 정의합니다.

Fallback Ladder



예시:

1. 기본 모델 호출 2. 같은 모델 재시도 3. 더 단순한 Prompt 또는 출력 축소 4. 허용된 대체 모델 5. 캐시된 이전 결과 또는 기능 축소 6. 사람 검토/나중에 다시 시도 안내

Fallback은 품질과 기능이 달라질 수 있으므로 사용자에게 숨기지 않습니다.

기능 축소 예

멀티모달 영수증 자동 처리 실패 -> 상호영/총액만 추출 -> 그래도 실패하면 수동 입력 화면 제공

대체 모델 주의

- 같은 Tool과 Structured Output을 지원하는지 확인합니다.
- model-specific 파라미터 지원 여부를 확인합니다.
- 품질과 가격 차이를 평가합니다.
- fallback이 사용되었다는 로그를 남깁니다.

7. 기본 실습: 로컬 Retry 함수 만들기

API 비용 없이 동작을 확인하기 위해 정해진 횟수만 실패하는 가짜 함수를 만듭니다.

```
# 첫 두 번은 실패하고 세 번째에 성공하는 함수를 retry wrapper로 실행합니다.
import time
from collections.abc import Callable
from dataclasses import dataclass
from typing import T
typeVar T = TypeVar("T") # TemporaryError 객체의 데이터 구조와 검증 규칙을 정의합니다.
class TemporaryError(RuntimeError):
    """재시도 가능한 일시 오류를 나타내는 실습용 예외입니다."""
# RetryReport 객체의 데이터 구조와 검증 규칙을 정의합니다.
@dataclass
class RetryReport:
    attempts: int # 실제 호출을 시도한 횟수입니다.
    waited_seconds: float # retry_call 처리 단계를 함수로 묶어 재사용합니다.
    def retry_call(operation: Callable[[], T], max_attempts: int = 3, sleep_function: Callable[[float], None] = time.sleep, ) -> tuple[T, RetryReport]:
        """TemporaryError만 backoff 후 재시도합니다."""
        # 남은 재시도 횟수와 오류 종류를 기준으로 계속할지 판단합니다.
        if max_attempts < 1: # 잘못된 입력을 조기에 중단해 이후 단계로 전파되지 않게 합니다.
            raise ValueError("max_attempts는 1 이상이어야 합니다.")
        # 전체 재시도 과정에서 기다린 시간을 누적합니다.
        total_wait = 0.0 # 각 항목을 순회하며 현재 조건에 해당하는지 확인합니다.
        for attempt in range(1, max_attempts + 1): # 실패할 수 있는 변환 또는 외부 호출을 예외 처리 범위에 넣습니다.
            try:
                value = operation()
            # 현재 단계의 검증된 결과를 호출한 쪽에 반환합니다.
            except TemporaryError: # 예상한 오류만 잡아 오류 종류에 맞는 결과로 바꿉니다.
                except TemporaryError: # 마지막 시도까지 실패했다면 더 기다리지 않고 예외를 다시 올립니다.
                    if attempt == max_attempts:
                        raise # 현재 재시도 전에 기다릴 시간을 계산합니다.
                        delay = calculate_backoff_seconds(attempt - 1)
                        total_wait += delay
                        print(f"{attempt}차 실패,{delay:.2f}초 후 재시도합니다.")
                        sleep_function(delay)
                    # for문 구조상 도달하지 않지만 타입 검사기를 위한 방어 코드입니다.
                    raise RuntimeError("도달할 수 없는 코드입니다.")
```

가짜 operation

```
# 테스트용 함수가 몇 번 호출되었는지 상태를 저장합니다.
attempt_counter = {"count": 0}
# flaky_operation 처리 단계를 함수로 묶어 재사용합니다.
def flaky_operation() -> str:
    """두 번 실패한 뒤 성공하는 실습용 함수입니다."""
    attempt_counter["count"] += 1
    # 남은 재시도 횟수와 오류 종류를 기준으로 계속할지 판단합니다.
    if attempt_counter["count"] < 3: # 잘못된 입력을 조기에 중단해 이후 단계로 전파되지 않게 합니다.
        raise TemporaryError("일시적 실패")
    # 현재 단계의 검증된 결과를 호출한 쪽에 반환합니다.
    return "success"
# 테스트 시간을 줄이기 위해 실제 sleep 대신 대기 시간을 출력만 합니다.
value, report = retry_call(flaky_operation, max_attempts=4, sleep_function=lambda seconds: print(f"[test sleep]{seconds:.2f}s"), )
print(value)
print(report)
assert value == "success"
assert report.attempts == 3
```

8. 연습 문제: 재시도 정책 판단

문제 1

API key가 잘못되어 인증 오류가 발생했습니다. exponential backoff로 다섯 번 재시도해야 하나요?

- ▼ 정답 확인

아닙니다. secret과 권한을 바꾸지 않으면 같은 오류가 반복됩니다. 즉시 중단하고 설정을 확인해야 합니다.

문제 2

429 Rate Limit에서 기다림 없이 0.1초마다 재시도하는 방식의 문제는 무엇인가요?

- ▼ 정답 확인

제한이 회복되기 전에 실패 요청을 반복해 상황을 악화시키고 비용/지연시간을 늘릴 수 있습니다. exponential backoff, jitter, Retry-After, 동시성 제한을 사용해야 합니다.

문제 3

SDK 기본 retry가 2회이고 애플리케이션 retry가 최대 3회라면 최악의 실제 시도 횟수는 어떻게 늘어날 수 있나요?

- ▼ 예시 정답

각 애플리케이션 시도 안에서 초기 요청과 SDK 재시도 2회가 일어나면 최대 3회입니다. 애플리케이션이 그 묶음을 3회 실행하면 최악의 경우 최대 9회 요청이 될 수 있습니다. 정확한 정책을 문서화하고 중복 retry를 피해야 합니다.

9. 추가 실습: Responses API Wrapper 구현

다음 코드는 실제 OpenAI SDK 예외 유형을 구분합니다. 실제 API 비용이 발생할 수 있습니다.

```
# Responses API 호출을 제한된 수동 retry로 감싸고, # 재시도 가능한 예외와 즉시 중단할 예외를 구분합니다.
import os
import random
import time
from dataclasses import dataclass
from typing import Any
from openai import (
    APIConnectionError,
    APIStatusError,
    APITimeoutError,
    AuthenticationError,
    BadRequestError,
    OpenAI,
    RateLimitError,
)
# 환경 변수로 모델을 교체할 수 있게 하며, 기본값은 실험 시점의 예시입니다.
MODEL = os.getenv("OPENAI_MODEL", "gpt-5.6")
# 수동 wrapper가 retry 횟수를 전부 관리하도록 SDK retry를 끕니다.
# API key는 코드에 쓰지 않고 OPENAI_API_KEY 환경 변수 또는 Colab Secrets에서 읽습니다.
client = OpenAI(max_retries=0, timeout=30.0)
# StableCallResult 객체의 데이터 구조와 검증 규칙을 정의합니다.
@dataclass
class StableCallResult:
    ok: bool
    # 검증 또는 실행이 성공했는지 표시합니다.
    output_text: str | None
    error_code: str | None
    # 실패 원인을 프로그램이 분기할 코드로 보관합니다.
    attempts: int
    # 실제 호출을 시도한 횟수입니다.
    latency_ms: float
    # API 지연시간입니다.
    request_id: str | None
    # 장애 추적을 API 요청 ID입니다.
    is_retryable_status: bool
    # 처리 단계를 함수로 묶어 재사용합니다.
    def is_retryable_status(status_code: int | None) -> bool:
        """HTTP 상태 코드가 수동 retry 대상인지 판단합니다."""
        # 필수 객체나 조회 결과가 없는 실패 경로를 먼저 처리합니다.
        if status_code is None:
            # 현재 단계의 검증된 결과를 호출한 쪽에 반환합니다.
            return status_code in {408, 409, 429}
        # Responses API 호출에 수동 retry와 상태 기록을 적용합니다.
        def create_response_with_retry(
            input_data: Any,
            max_attempts: int = 3,
        ) -> StableCallResult:
            """일시적 오류만 backoff 후 재시도합니다."""
            # wall clock 변경의 영향을 덜 받는 perf_counter로 경과 시간을 측정합니다.
            started = time.perf_counter()
            last_error_code = "unknown"
            # request ID는 장애 문의와 추적에 유용하므로 성공/실패 로그에 남깁니다.
            last_request_id: str | None = None
            # 마지막 실패 응답의 요청 ID입니다.
            # 각 항목을 순회하며 현재 조건에 해당하는지 확인합니다.
            for attempt in range(1, max_attempts + 1):
                # 실패할 수 있는 변환 또는 외부 호출을 예외 처리 범위에 넣습니다.
                try:
                    # 이 줄에서 실제 API 요청이 발생합니다.
                    # 반복 실행 시 token 비용과 rate limit을 확인합니다.
                    response = client.responses.create(
                        model=MODEL,
                        # 호출에 사용할 모델 ID를 전달합니다.
                        input=input_data,
                        # 모델이 처리할 사용자 입력 또는 누적 item을 전달합니다.
                        max_output_tokens=500,
                        # 비용과 응답 길이를 제어할 최대 출력 토큰 수입니다.
                    )
                    # 현재 단계의 검증된 결과를 호출한 쪽에 반환합니다.
                    return StableCallResult(
                        ok=True,
                        # 현재 단계의 성공 여부를 전달합니다.
                        # Tool 결과 dict를 모델에 전달할 JSON 문자열로 직렬화합니다.
                        output_text=response.output_text,
                        error_code=None,
                        # 프로그램이 분기할 오류 코드를 전달합니다.
                        attempts=attempt,
                        # 호출 종료 시각과 시작 시각의 차이를 밀리초로 변환합니다.
                        latency_ms=round((time.perf_counter() - started) * 1000, 1),
                        request_id=response.request_id,
                    )
                    # 인증/Bad Request는 입력이나 설정을 고치지 않으면 반복되므로 즉시 중단합니다.
                    except (AuthenticationError, BadRequestError) as error:
                        # 요청 설정을 바꾸지 않으면 같은 오류가 반복되므로 즉시 중단합니다.
                        return StableCallResult(
                            ok=False,
                            # 현재 단계의 성공 여부를 전달합니다.
                            # Tool 결과 dict를 모델에 전달할 JSON 문자열로 직렬화합니다.
                            output_text=None,
                            error_code=type(error).__name__,
                            # 프로그램이 분기할 오류 코드를 전달합니다.
                            attempts=attempt,
                            # 호출 종료 시각과 시작 시각의 차이를 밀리초로 변환합니다.
                            latency_ms=round((time.perf_counter() - started) * 1000, 1),
                            # SDK 버전이나 선택 필드가 없어도 안전하게 기본값을 사용합니다.
                            request_id=getattr(error, "request_id", None),
                        )
                        # 429는 일시적일 수 있지만 무한 재시도하지 않고 횟수/deadline을 제한합니다.
                        except RateLimitError as error:
                            last_error_code = "rate_limit"
                            # SDK 버전이나 선택 필드가 없어도 안전하게 기본값을 사용합니다.
                            last_request_id = getattr(error, "request_id", None)
                            # 예상한 오류만 잡아 오류 종류에 맞는 결과로 바꿉니다.
                            except (APIConnectionError, APITimeoutError) as error:
                                last_error_code = type(error).__name__
                                # SDK 버전이나 선택 필드가 없어도 안전하게 기본값을 사용합니다.
                                last_request_id = getattr(error, "request_id", None)
                                # 예상한 오류만 잡아 오류 종류에 맞는 결과로 바꿉니다.
                                except APIStatusError as error:
                                    # SDK 버전이나 선택 필드가 없어도 안전하게 기본값을 사용합니다.
                                    last_request_id = getattr(error, "request_id", None)
                                    # 남은 재시도 횟수와 오류 종류를 기준으로 계속할지 판단합니다.
                                    if not is_retryable_status(error.status_code):
                                        # 현재 단계의 검증된 결과를 호출한 쪽에 반환합니다.
                                        return StableCallResult(
                                            ok=False,
                                            # 현재 단계의 성공 여부를 전달합니다.
                                            # Tool 결과 dict를 모델에 전달할 JSON 문자열로 직렬화합니다.
                                            output_text=None,
                                            error_code=f"http_{error.status_code}",
                                            # 프로그램이 분기할 오류 코드를 전달합니다.
                                            attempts=attempt,
                                            # 호출 종료 시각과 시작 시각의 차이를 밀리초로 변환합니다.
                                            latency_ms=round((time.perf_counter() - started) * 1000, 1),
                                            request_id=last_request_id,
                                            last_error_code = f"http_{error.status_code}"
                                        )
                                        # 마지막 시도까지 실패했다면 더 기다리지 않습니다.
                                        if attempt == max_attempts:
                                            break
                                    # 현재 재시도 전에 기다릴 시간을 계산합니다.
                                    delay = calculate_backoff_seconds(attempt - 1)
                                    print(f"attempt={attempt} error={last_error_code} " f"wait={delay:.2f}s")
                                    # 실제 대기 전에 마지막 시도인지 확인하며, 테스트에서는 sleep 함수를 주입해 시간을 줄일 수 있습니다.
                                    time.sleep(delay)
                                    # 현재 단계의 검증된 결과를 호출한 쪽에 반환합니다.
                                    return StableCallResult(
                                        ok=False,
                                        # 현재 단계의 성공 여부를 전달합니다.
                                        # Tool 결과 dict를 모델에 전달할 JSON 문자열로 직렬화합니다.
                                        output_text=None,
                                        error_code=last_error_code,
                                        # 프로그램이 분기할 오류 코드를 전달합니다.
                                        attempts=max_attempts,
                                        # 호출 종료 시각과 시작 시각의 차이를 밀리초로 변환합니다.
                                        latency_ms=round((time.perf_counter() - started) * 1000, 1),
                                        request_id=last_request_id,
                                    )
```

### 실행 예

```
result = create_response_with_retry( "배송 문의를 delivery, refund, account 중 하나로 분류하고 라벨만 출력하세요." )
print(result)
```

### 이 wrapper에 추가할 수 있는 항목

- Retry-After header 반영
- 동시성 제한
- circuit breaker
- fallback model
- Prompt 버전과 token 로그
- 요청별 idempotency 정책

## 10. 실제 활용과 다음 강의 연결

안정적 호출 wrapper는 7~9장의 모든 API 호출에 적용할 수 있습니다.

Structured Output 호출 Tool Calling 1차/2차 호출 멀티모달 이미지 분석 Reasoning 비교 실험

다음 10-2강에서는 호출 성공 여부뿐 아니라 token, cached token, latency, 예상 비용을 기록합니다. 또한 긴 고정 Prompt를 반복 사용할 때 Prompt Caching을 활용하는 구조를 학습합니다.

## 11. 자주 생기는 오해

오해 1. 모든 오류는 재시도하면 언젠가 해결됩니다.

인증/Bad Request/권한 오류는 설정이나 입력을 고쳐야 합니다.

오해 2. SDK가 retry하므로 timeout 설정은 필요 없습니다.

사용자가 기다릴 수 있는 시간과 업무 SLA에 맞는 timeout이 필요합니다.

오해 3. 재시도 횟수를 늘리면 안정성이 항상 좋아집니다.

지연시간, 비용, 서버 부하, 중복 부작용이 늘 수 있습니다.

오해 4. fallback은 사용자에게 숨겨도 됩니다.

가능이나 품질이 달라진다면 적절히 안내하고 로그에 기록해야 합니다.

오해 5. 조회와 상태 변경 요청을 같은 방식으로 재시도해도 됩니다.

상태 변경은 idempotency와 성공 여부 확인이 없으면 중복 실행될 수 있습니다.

## 12. 이해도 점검

- 재시도 가능한 오류와 불가능한 오류를 각각 두 가지 적어 보세요.
- exponential backoff에 jitter를 더하는 이유는 무엇인가요?
- SDK retry와 애플리케이션 retry를 함께 사용할 때 무엇을 주의해야 하나요?
- timeout을 태스크별로 다르게 설정할 수 있는 이유는 무엇인가요?
- fallback ladder의 예를 세 단계로 작성해 보세요.
- 부작용 Tool retry에 idempotency가 필요한 이유는 무엇인가요?

▼ 정답 확인

1. 재시도 가능: 연결 오류, timeout, 429, 5xx, 재시도 불가: 인증 실패, Bad Request, 권한 실패 등입니다.
2. 많은 client가 동시에 같은 간격으로 재시도해 다시 부하를 만드는 현상을 줄이기 위해서입니다.
3. retry 횟수가 곱해져 실제 요청 수와 지연시간이 크게 늘 수 있으므로 각 층의 횟수를 문서화해야 합니다.
4. 단순 분류와 Reasoning/멀티모달 작업의 정상 처리 시간이 다르기 때문입니다.
5. 기본 모델 재시도 -> 단순화된 Prompt/대체 모델 -> 수동 처리 안내처럼 작성할 수 있습니다.
6. timeout 뒤 실제 실행 여부가 불명확할 때 같은 작업이 중복 실행되는 것을 막기 위해서입니다.

13. 이번 강의 요약

- API 오류를 retryable과 non-retryable로 구분해야 합니다.
- SDK는 일부 연결/408/409/429/5xx 오류를 기본 재시도할 수 있습니다.
- 수동 retry를 추가할 때 SDK retry와 중복되지 않게 정책을 정합니다.
- exponential backoff와 jitter로 재시도 간격을 늘립니다.
- 인증/Bad Request/권한 오류는 설정을 바꾸지 않은 재시도를 하지 않습니다.
- fallback은 기능 축소, 대체 모델, 사람 검토로 이어지는 명시적 정책입니다.
- 상태 변경 Tool에는 idempotency가 필요합니다.
- 다음 강의에서는 Prompt Caching과 token/cost/latency logging을 구현합니다.